
LAB3: Product Testing Study

Ignacio González Peris

20 de febrero de 2026

Abstract

En este laboratorio se aborda un caso de *Product Testing Study* con dos objetivos: (i) entrenar y evaluar un modelo de predicción de fallos (*failure*) bajo un escenario de generalización exigente, y (ii) diagnosticar *data drift* entre los conjuntos de entrenamiento y prueba mediante *validación adversarial*.

Se parte de un `train.csv` con 26.570 filas (incluyendo el target) y un `test.csv` con 20.775 filas (sin target). El análisis exploratorio revela desbalance de clases en *failure* y, especialmente, un desfase crítico de `product_code` (sin solape entre train y test), lo que obliga a que el modelo generalice a partir de atributos y mediciones. Se aplica ingeniería de características: eliminación de columnas no generalizables, *one-hot encoding* consistente, indicadores de *missingness* e imputación robusta por mediana.

Para la tarea de predicción se utiliza un `GradientBoostingClassifier` evaluado con validación cruzada estratificada, obteniendo un rendimiento AUC-ROC modesto. Para la tarea de *drift* se entrena un clasificador para distinguir train vs test, alcanzando AUC-ROC perfecto, lo que evidencia un *drift* severo explicado casi totalmente por los atributos del producto.

Keywords: MLops, Product Testing, EDA, Feature Engineering, AUC-ROC, Data Drift, Adversarial Validation

Contents

1	Estructura del notebook y carga de datos	3
2	Análisis Exploratorio de Datos (EDA)	3
2.1	Distribución del objetivo	3
2.2	Desfase crítico en <code>product_code</code>	3
2.3	Valores faltantes	3
3	Ingeniería de Características	3
3.1	Eliminación de columnas no generalizables	3
3.2	Codificación de variables categóricas	4
3.3	Indicadores de missingness	4
3.4	Imputación por mediana	4
3.5	Resultado del pipeline	4
4	Tarea 1: Predicción de fallos	4
4.1	Modelo: <code>GradientBoostingClassifier</code>	4
4.2	Evaluación: validación cruzada estratificada	4
4.3	Predicciones en test	5
5	Tarea 2: Validación adversarial (detección de Data Drift)	5
5.1	Concepto y procedimiento	5
5.2	Interpretación del AUC adversarial	5
5.3	Resultados y explicación	5
5.4	Conclusión conjunta (Tarea 1 + Tarea 2)	6
6	Resumen de resultados	6

1 Estructura del notebook y carga de datos

El notebook LAB2-DataDrift.ipynb se organiza en cuatro secciones: (0) carga de datos, (1) análisis exploratorio (EDA), (2) ingeniería de características y (3) tarea de predicción de fallos; además, se incluye una (4) validación adversarial para detección de *data drift*.

Se cargan dos archivos CSV:

- **train.csv**: 26.570 filas y 26 columnas, incluyendo la variable objetivo *failure*.
- **test.csv**: 20.775 filas y 25 columnas, sin *failure*.

2 Análisis Exploratorio de Datos (EDA)

2.1 Distribución del objetivo

La variable *failure* es binaria y presenta desbalance:

- **Clase 0 (sin fallo)**: 20.921 muestras ($\approx 78,74\%$).
- **Clase 1 (fallo)**: 5.649 muestras ($\approx 21,26\%$).

Este desbalance hace que la exactitud (*accuracy*) sea poco informativa; por ello se adopta **AUC-ROC** como métrica principal, al evaluar capacidad de ranking independientemente del umbral.

2.2 Desfase crítico en *product_code*

Se observa un cambio estructural entre train y test:

Conjunto	Códigos de producto
Train	A, B, C, D, E
Test	F, G, H, I

No existe solape de códigos entre ambos conjuntos. Esto implica que el modelo no debe depender de identidades de producto, sino generalizar mediante *loading*, *attribute_** y *measurement_**.

2.3 Valores faltantes

Los valores ausentes se concentran en mediciones, con un patrón progresivo: desde *measurement_3* (381 ausentes) hasta *measurement_17* (2.284 ausentes). Además, *loading* tiene 250 valores nulos. No se observan nulos en *attribute_** ni en *failure*.

3 Ingeniería de Características

La preparación de datos se comparte entre tareas y busca maximizar generalización a productos nuevos.

3.1 Eliminación de columnas no generalizables

- Se elimina *product_code*, ya que train y test no comparten códigos y su inclusión induciría reglas espurias.
- Se elimina *id*, por ser un identificador sin información semántica del producto.

3.2 Codificación de variables categóricas

Las variables `attribute_0` y `attribute_1` contienen valores tipo `material_k`. Se aplica **one-hot encoding** sobre el conjunto combinado `train+test` para asegurar el mismo espacio de características en ambos conjuntos.

3.3 Indicadores de missingness

Para cada columna con nulos se crea una variable binaria (por ejemplo, `measurement_3_missing`) que indica si el valor original era nulo. El patrón de ausencia puede ser informativo si refleja condiciones anómalas (p. ej. sensores que no registran).

3.4 Imputación por mediana

Los valores nulos restantes se imputan con la **mediana** calculada *solo en train* y aplicada a `train` y `test`, evitando *data leakage* y manteniendo robustez ante outliers.

3.5 Resultado del pipeline

Tras el preprocesado se obtiene un conjunto final de **41 características** sin valores nulos.

4 Tarea 1: Predicción de fallos

4.1 Modelo: GradientBoostingClassifier

Se entrena un `GradientBoostingClassifier` con:

```
— n_estimators = 300
— learning_rate = 0.05
— max_depth = 4
— subsample = 0.8
```

Este modelo es apropiado por su robustez con variables mixtas, su capacidad para capturar no linealidades y su bajo requisito de normalización.

4.2 Evaluación: validación cruzada estratificada

Se aplica `StratifiedKFold` con 5 particiones para mantener la proporción de fallos en cada fold. Los resultados AUC-ROC por fold son:

```
Fold 1: AUC-ROC = 0.5755
Fold 2: AUC-ROC = 0.5734
Fold 3: AUC-ROC = 0.5783
Fold 4: AUC-ROC = 0.5619
Fold 5: AUC-ROC = 0.5740
```

Media AUC-ROC: 0.5726 ± 0.0056

El $AUC \approx 0.57$ es modesto y coherente con la dificultad del escenario: los productos en `test` son intrínsecamente nuevos (códigos F-I), por lo que el modelo solo puede apoyarse en atributos y mediciones.

4.3 Predicciones en test

Se generan probabilidades de fallo para cada `id` del conjunto de prueba, creando el dataframe `submission` con columnas `id` y `failure`. Las probabilidades oscilan aproximadamente entre 0.05 y 0.81, con media cercana a 0.22, alineada con la tasa de fallos del entrenamiento.

5 Tarea 2: Validación adversarial (detección de Data Drift)

5.1 Concepto y procedimiento

La **validación adversarial** detecta *data drift* entrenando un clasificador que distingue si una fila proviene de train o de test:

1. Se elimina `failure` de train y `id` de ambos conjuntos.
2. Se combinan train y test en un único dataframe.
3. Se crea un target artificial: 0 para train y 1 para test.
4. Se entrena un clasificador y se evalúa con AUC-ROC.

Nota sobre `product_code`: también se elimina en este análisis. Si se incluyera, el clasificador obtendría AUC=1.0 de manera trivial (A–E vs F–I), sin aportar información útil sobre drift en variables predictivas.

5.2 Interpretación del AUC adversarial

- Si hay drift significativo: AUC-ROC alto ($\gg 0.5$, cercano a 1.0). El modelo de fallos probablemente generaliza peor al test.
- Si no hay drift: AUC-ROC cercano a 0.5. Train y test son indistinguibles y el modelo debería generalizar mejor.

5.3 Resultados y explicación

Se obtienen AUC-ROC adversariales perfectos:

AUC-ROC adversarial (cada fold): [1.0, 1.0, 1.0, 1.0, 1.0]

Media: 1.0000 ± 0.0000

Esto confirma **data drift severo**. El análisis de importancias indica que el drift está dominado por atributos del producto:

Table 1 – Principales variables responsables del drift (importancia del clasificador adversarial).

Característica	Importancia
<code>attribute_3</code>	0.283
<code>attribute_2</code>	0.231
<code>attribute_1_material_8</code>	0.222
<code>attribute_1_material_7</code>	0.181
<code>attribute_1_material_6</code>	0.049
<code>attribute_0_material_7</code>	0.034
<code>loading, measurement_*</code>	~ 0.000

El drift es, por tanto, **estructural**: los productos de test presentan materiales/atributos distintos a los vistos en train, mientras que las mediciones numéricas mantienen distribuciones similares.

5.4 Conclusión conjunta (Tarea 1 + Tarea 2)

Aunque existe drift severo en atributos, las variables continuas de medición no muestran drift relevante. Esto ayuda a explicar por qué el modelo de fallos, pese a enfrentar productos no vistos, puede producir predicciones razonables, aunque con rendimiento limitado.

6 Resumen de resultados

Table 2 – Resumen cuantitativo de resultados del laboratorio.

Tarea	Métrica	Resultado
Task 1: Predicción de fallos	AUC-ROC (CV 5-fold)	0.5726 ± 0.0056
Task 2: Detección de drift	AUC-ROC adversarial	1.0000 ± 0.0000